

企业级技能平台

架构设计、核心功能、开发规范与物理目录映射技术手册

目录 (TABLE OF CONTENTS)

- 1. 系统架构图与整体物理层设计 Page 2
- 2. 项目核心代码三级子目录细化说明 Page 3
- 3. 核心业务概念与端到端流水线 Page 4
- 4. 平台核心功能与业务域详解 Page 5
- 5. 容器部署、数据库迁移与开发规范 Page 6

系统架构图与核心设计平面

生成日期: 2026年6月

Enterprise Skill Platform Architecture & Core Planes

治理平面 (Governance Plane)

身份权限 (Identity)

JWT/LDAP 整合体系, 通过 RbacGuard、JwtAuthGuard 等控制资源访问权限。

组织架构 (Organization)

支持多租户与组织隔离, 对接企业级部门及映射服务。

审计控制 (Audit & Security)

对所有敏感操作、沙箱逻辑及 API 调用进行防篡改审计记录。

智能平面 (Intelligence Plane)

AI 编排器 (AI Orchestrator)

大模型解析意图, 推荐和动态组合最符合的 Skill 业务流。

网页动作智能体 (Browser NL Agent)

通过视觉/语义直接在网页中执行探索、点击和数据提取。

代码生成智能体 (Codegen Agent)

自动产生受控 Python/JS 脚本并在沙箱中执行处理逻辑。

执行控制面 (Execution Control)

控制平面主核 (Control Plane)

任务下行调度与状态机, 协调接管及**周期任务定时器 (Scheduler) **。

会话协调器 (Session Broker)

负责远程有状态资源 (如 Chrome) 的并发锁定、分配及租约续期。

运行时与能力域 (Runtimes & Domains)

浏览器能力域 (Browser Domain)

录制引擎服务, 配合 `browser-worker` 在远程 VNC 运行。

文档模板渲染 (Document Domain)

集成 Carbone 实现 Word/Excel 数据注入与多格式高还原 PDF 输出。

持久工作流 (Temporal Worker)

长生命周期逻辑。确保网络断连等故障发生后状态自动恢复。

物理层一级结构映射

apps/backend/

后端微服务集群。包含底层治理面、核心控制面、南向运行时与专门业务域。

apps/frontend/

浏览器前端。`user-web` 用于操作和调度, `portal` 供管理员设计和管理 Skill。

packages/

内部依赖包。`backend-contracts` 管理微服务契约, `user-core` 承载前端 API SDK。

docker/

统一容器底座。包含各服务的 compose 配置与 SQL 迁移脚本。

项目核心代码三级子目录细化说明

生成日期: 2026年6月

Detailed Mapping of Project Code Subdirectories (Level 3)

代码模块职责说明（三级子目录）

三级物理目录路径	所属逻辑层	主要职责说明
apps/backend/execution-control/control-plane	执行控制面 (Control Plane)	任务生命周期编排、状态机控制、审批拦截、接管分发与**定时任务调度 (Scheduler)**。
apps/backend/execution-control/session-broker	执行控制面 (Broker)	负责有状态资源（如 Chrome 会话）的锁定、并发限制、配额分配与租约心跳监控。
apps/backend/runtimes/browser-worker	执行运行时 (Worker)	接收浏览器自动化执行指令，拉起 Chrome headless 操控页面并执行抓取或填写动作。
apps/backend/runtimes/sandbox-worker	执行运行时 (Worker)	动态 JavaScript/Python 脚本受控执行器，在高度隔离的沙箱环境运行，杜绝宿主逃逸。
apps/backend/runtimes/temporal-worker	执行运行时 (Temporal)	对接 Temporal Cluster，用于执行有状态、长周期的 DAG 工作流。
apps/backend/intelligence/ai-orchestrator	智能计算面 (Orchestrator)	意图解析核心，基于大模型语义向量转换，生成 Skill 参数映射并做置信度路由。
apps/backend/intelligence/browser-nl-agent	智能计算面 (Agent)	利用视觉与语言推理模型动态在网页中进行 DOM 识别与探索，无需录制即可执行网页动作。
apps/backend/capabilities/browser-domain	业务领域层 (Browser)	负责管理浏览器动作录制模版定义、XPath 提取以及设计态的元数据存储。
apps/backend/capabilities/document-domain	业务领域层 (Document)	Carbone 引擎的高级封装，处理报表配置模版的增删改查，完成数据字段注入渲染。
apps/backend/governance/identity-access	企业治理层 (Auth)	系统统一的 IAM 方案。支持 LDAP 绑定、系统 RBAC 角色组定义与 JwtAuthGuard 鉴权中间件。
apps/backend/governance/organization	企业治理层 (Org)	管理多租户组织单元映射，处理跨组织的数据隔离与逻辑边界。
apps/frontend/user-web	展现层 (Frontend)	普通用户端门户，提供 Skill 可视化表单、执行历史、以及定时任务的 CRUD 管理。
packages/backend-contracts	契约包层 (Packages)	存储微服务强契约声明（如各微服务间 Event 强类模型），确保微服务通讯一致性。
packages/user-core	公共包层 (Packages)	前端通用网络请求封装 SDK、公共断言验证库与参数提取工具包。

核心业务概念与执行流水线

生成日期: 2026年6月

Core Concepts and End-to-End Execution Pipeline

1. 四大核心物理实体

Work Unit

工作单元

系统最基础的可执行单元。如点击元素或查询数据库。有明确输入输出并可被独立重试。

Workflow

工作流

将多个 Work Unit 进行逻辑关系串联（串行、条件分支、循环）形成的业务场景模板。

Release

发布资产

设计时到运行时的桥梁。设计草稿必须编译成只读 Release 经审批后才可生效执行。

Skill

可用技能

暴露给用户消费的最终对象。将特定的 Release 与 API 路由、执行表单及参数映射绑定。

2. 端到端执行流程机制

1

自然语言意图输入与推荐 (Intent Matching)

用户在前端输入意图，`ai-orchestrator` 解析语义并匹配最佳技能。

2

参数补全与动态校验 (Parameter Generation)

AI 自动提取入参实体并校验。缺失关键实体时向用户发出交互提问补齐。

3

任务分发与异步执行 (Southbound Dispatching)

控制面实例化参数后，由南向 Worker（如 browser-worker 等）执行物理流程。

4

人工干预接管 (Human Takeover)

执行中若遇到滑块风控，暂停工作流并支持用户在 Web 直接 VNC 接管，完成后恢复。

平台核心功能与业务域详解

生成日期: 2026年6月

Key Features and Business Capability Domains

NLP INTENT

智能意图识别与推荐

融合 NLP 与意图分类，将模糊指令翻译为结构化执行参数。支持语义模糊匹配与实时交互式补齐，用户体验简单直观。

SCHEDULER

分布式周期任务调度 (Prisma + PostgreSQL)

支持配置技能定时任务。通过后台 `**`SELECT FOR UPDATE SKIP LOCKED`**` 悲观锁抢占式 Tick，确保控制面横向高可用扩容时不重复触发任务。支持在 `published-skills` 页面做 CRUD 管理和手动一键触发。

VNC REPLAY

有状态浏览器自动化与动作录制

基于 Puppeteer & VNC 远程会话镜像，支持用户可视化录制动作。运行态支持重试。风控阻断时，任务挂起并支持用户在 Web 端一键接管控制。

CARBONE

模版注入与高还原文档填充

集成 carbone-engine，渲染 Word/Excel 模版变量，数据以 JSON 注入生成高保真 PDF 或表格报表供用户直接下载。

TEMPORAL

异构 workflow 状态编排与容灾

借助 Temporal 框架维护长周期 workflow。服务崩溃升级或网络意外断开时自动持久化断点，支持故障节点无差错断点续行。

容器部署、数据库迁移与开发规范

生成日期: 2026年6月

Developer Onboarding and Quality Constraints

1. DOCKER 容器开发上手

启动全栈开发环境 (Smart Start)

防止在 Git worktree 场景下运行旧代码或挂载错乱路径，禁止直接运行 `docker compose`。必须使用统一的 smart 脚本启动：

```
./docker/start-smart.sh docker-compose.base.yml up -d
```

数据库迁移约束 (Prisma Rules)

控制面与平台共享底层的 PostgreSQL，但各拥有局部定义。****禁止****直接执行 `prisma db push`（防止 drop 掉对方服务的数据表），必须使用工具更新：

```
./docker/scripts/apply-latest-db-schema.sh
```

2. 代码组织与单文件大小规范

为保证代码的可测试性与长期维护性，系统建立了严格的物理结构边界和行数约束（限普通业务逻辑文件，协议及常量文件除外）：

推荐警戒线 (Should Split) 800 行

文件达到此行数时需前置评估职责下沉与拆分。

强制拆分线 (Must Split) 1200 行

普通逻辑文件超出 1200 行必须重构拆分成细粒度模块。

模块命名后缀规范 *.service.ts / *.mapper.ts

根据职责合理划分后缀，防止万能 Service 文件堆积。